

Università degli Studi di Salerno

Corso di Laurea Magistrale in Ingegneria Informatica

Segmentazione dei filamenti solari

Relazione del progetto di Machine Learning

Gruppo

2026_MLrec_gr02

Componenti

Francesco Peluso

Anno accademico 2025/2026

1 Introduzione

L'obiettivo di questo progetto è identificare i filamenti solari nelle immagini del Sole mediante tecniche di Machine Learning. Il problema è formulato come un'attività di **segmentazione semantica binaria**: data un'immagine solare, il modello deve produrre una maschera binaria in cui ogni pixel viene classificato come *filamento* oppure *sfondo*.

L'analisi delle immagini di addestramento evidenzia alcune difficoltà rilevanti. I filamenti solari si presentano generalmente come strutture scure, sottili e irregolari sul disco solare. Il loro contrasto rispetto alle regioni circostanti può essere basso, mentre forma e dimensioni possono variare considerevolmente tra immagini diverse. Inoltre, la superficie solare contiene altre strutture e rumore di fondo che possono avere caratteristiche visive simili a quelle di un filamento.

Un altro aspetto importante è il forte sbilanciamento tra le due classi. In genere, i filamenti occupano solo una piccola porzione dell'immagine completa, mentre la maggior parte dei pixel appartiene allo sfondo. Per questo motivo, identificare correttamente le regioni dei filamenti è più importante che massimizzare semplicemente l'accuratezza della classificazione a livello di pixel.

Il progetto si concentra quindi su modelli capaci di combinare l'estrazione di caratteristiche di alto livello con la conservazione delle informazioni spaziali fini. La qualità della segmentazione ottenuta viene valutata principalmente mediante il **coefficiente di Dice**, che misura la sovrapposizione tra la maschera binaria predetta e la corrispondente maschera di riferimento (*ground truth*).

2 Approccio proposto

2.1 Architettura della rete

L'approccio principale considerato per questo progetto si basa su una **rete neurale convoluzionale U-Net**, un'architettura progettata specificamente per la segmentazione delle immagini.

Le architetture CNN studiate per la classificazione delle immagini non sono direttamente adatte al nostro problema. Una rete di classificazione estrae progressivamente le caratteristiche da un'immagine e produce infine una predizione per l'intero ingresso, come un'etichetta di classe. Nella segmentazione semantica, invece, è importante anche la posizione delle caratteristiche rilevate, poiché la rete deve assegnare una classe a ogni pixel dell'immagine.

U-Net affronta questo problema mediante due percorsi principali. Il primo estrae progressivamente caratteristiche di livello più elevato e ne riduce la risoluzione spaziale, in modo analogo a una CNN standard. Il secondo aumenta progressivamente la risoluzione di tali caratteristiche fino a ottenere una maschera di segmentazione.

Una caratteristica importante di U-Net è l'impiego di **connessioni di salto** (*skip connections*) tra diverse parti della rete. Queste connessioni consentono di riutilizzare, durante la costruzione della maschera di uscita, le caratteristiche che contengono informazioni spaziali fini. Ciò è particolarmente utile nel nostro problema, poiché i filamenti solari possono essere sottili, irregolari e difficili da distinguere dalla superficie solare circostante.

2.2 Vantaggi e limitazioni

I principali vantaggi di questo approccio sono:

- U-Net è progettata specificamente per la segmentazione delle immagini a livello di pixel;
- si basa principalmente su operazioni convoluzionali, già adatte all'estrazione di caratteristiche spaziali dalle immagini;
- le connessioni di salto aiutano a conservare le informazioni relative alle strutture piccole e sottili;
- l'architettura è relativamente semplice e può essere modificata per valutare sperimentalmente configurazioni diverse.

È necessario considerare anche alcune limitazioni:

- la riduzione della risoluzione spaziale può causare la perdita di informazioni relative ai filamenti molto sottili;
- l'elaborazione di immagini ad alta risoluzione richiede una maggiore quantità di memoria della GPU;
- il forte sbilanciamento tra i pixel appartenenti ai filamenti e quelli appartenenti allo sfondo può rendere più difficile l'addestramento;
- strutture dall'aspetto simile ai filamenti possono generare predizioni false positive.

2.3 Approcci alternativi

Come alternativa, si potrebbe considerare una **U-Net residua** (*Residual U-Net*), introducendo blocchi residui e connessioni di salto all'interno della rete. Ciò consentirebbe di analizzare, nel contesto della segmentazione delle immagini, le tecniche di apprendimento residuo studiate durante il corso.

Un'altra possibilità consiste nell'utilizzare una **CNN preaddestrata** come parte dell'architettura U-Net e applicare il *transfer learning* e il *fine-tuning*. Questo approccio potrebbe essere utile soprattutto se l'insieme di addestramento disponibile non fosse sufficientemente ampio da consentire l'apprendimento da zero di caratteristiche visive robuste.

Si potrebbero considerare anche architetture di segmentazione più avanzate. Tuttavia, U-Net rappresenta un punto di partenza adeguato grazie alla sua architettura relativamente semplice e all'impiego diretto di strati convoluzionali e connessioni di salto.

3 Architettura U-Net

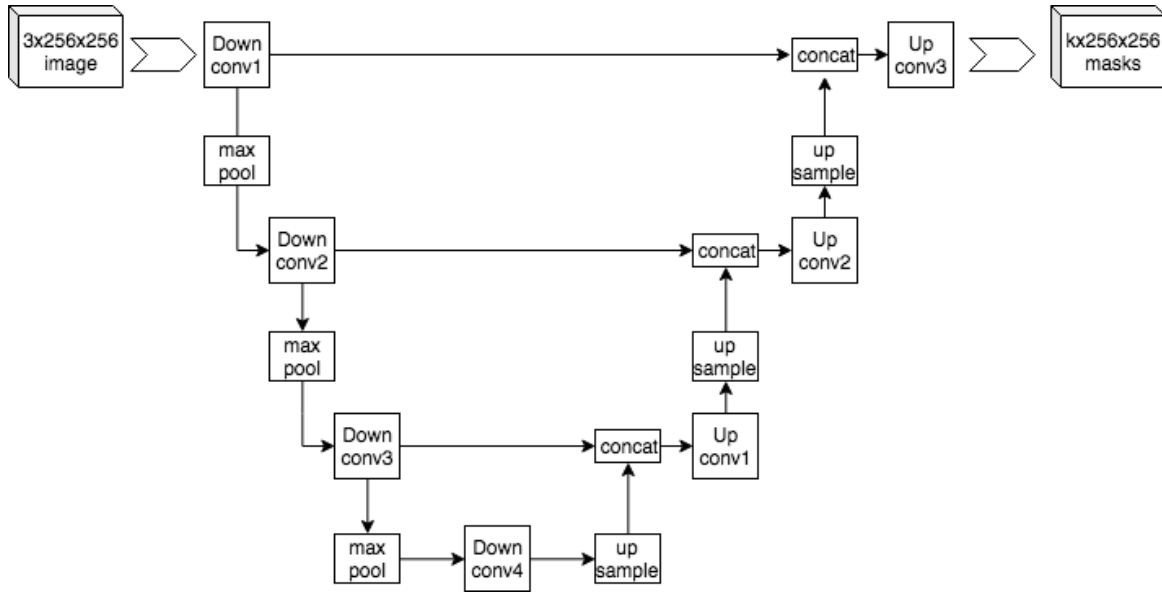


Figura 1: Struttura generale dell'architettura U-Net. L'encoder riduce progressivamente la risoluzione spaziale, mentre il decoder ricostruisce la mappa di segmentazione. Le connessioni di salto trasferiscono le caratteristiche ad alta risoluzione tra gli stadi corrispondenti.

Fonte: Mehrdad Yazdani – Opera propria, CC BY-SA 4.0,
<https://commons.wikimedia.org/w/index.php?curid=81055729>

U-Net è un'architettura di rete neurale convoluzionale originariamente progettata per la segmentazione delle immagini. A differenza di una CNN standard utilizzata per la classificazione, che produce un'unica predizione per l'intera immagine in ingresso, U-Net produce una predizione per ogni pixel. Pertanto, data un'immagine in ingresso di dimensioni $H \times W$, la rete produce una mappa di segmentazione con le stesse dimensioni spaziali.

L'architettura segue una struttura **encoder-decoder** e presenta una caratteristica forma simmetrica, dalla quale deriva il nome U-Net. Può essere suddivisa in tre componenti principali: l'encoder, il collo di bottiglia (*bottleneck*) e il decoder.

3.1 Encoder

L'**encoder** è la prima parte della rete e opera in modo simile a una CNN tradizionale. Il suo scopo è estrarre progressivamente caratteristiche sempre più complesse dall'immagine in ingresso.

Ogni stadio dell'encoder applica operazioni convoluzionali seguite da funzioni di attivazione non lineari. Le operazioni di sottocampionamento (*downsampling*) riducono progressivamente la risoluzione spaziale delle mappe delle caratteristiche, aumentandone al contempo il numero di canali.

Un esempio semplificato è:

$$H \times W \times C \rightarrow \frac{H}{2} \times \frac{W}{2} \times 2C \rightarrow \frac{H}{4} \times \frac{W}{4} \times 4C \rightarrow \dots \quad (1)$$

Durante il passaggio dell'immagine attraverso l'encoder, la rete passa quindi da informazioni spaziali ad alta risoluzione a caratteristiche più astratte. In questo modo può riconoscere strutture e configurazioni utili per distinguere i filamenti solari dalla superficie solare circostante.

Tuttavia, questo processo provoca anche una perdita progressiva di informazioni spaziali. Ciò non costituisce un problema rilevante per la classificazione delle immagini, in cui è richiesta soltanto la presenza di un oggetto o di una classe, ma diventa importante nella segmentazione, in cui occorre recuperare la posizione esatta di ogni struttura rilevata.

3.2 Collo di bottiglia

Il **collo di bottiglia** (*bottleneck*) rappresenta la parte più profonda di U-Net e collega l'encoder al decoder. In questo stadio, le mappe delle caratteristiche presentano la risoluzione spaziale più bassa, ma contengono la rappresentazione di più alto livello estratta dalla rete.

Il suo ruolo è fornire al decoder una rappresentazione compatta dell'immagine in ingresso, contenente le informazioni necessarie a determinare quali strutture possano corrispondere a filamenti solari.

3.3 Decoder

Il **decoder** esegue la trasformazione spaziale opposta rispetto all'encoder. A partire dalla rappresentazione a bassa risoluzione prodotta dal collo di bottiglia, aumenta progressivamente la risoluzione spaziale delle mappe delle caratteristiche.

Questo processo può essere rappresentato come segue:

$$\frac{H}{2^n} \times \frac{W}{2^n} \rightarrow \dots \rightarrow \frac{H}{2} \times \frac{W}{2} \rightarrow H \times W. \quad (2)$$

Lo scopo del decoder è quindi trasformare le caratteristiche estratte dall'encoder in una mappa di segmentazione a livello di pixel. Uno strato convoluzionale finale produce un valore di uscita per ogni pixel. Nel caso della segmentazione binaria, tali valori possono essere convertiti in probabilità di appartenenza a un filamento mediante una funzione di attivazione sigmoide.

3.4 Connessioni di salto

Una semplice architettura encoder-decoder risentirebbe comunque della perdita di informazioni spaziali causata dal sottocampionamento. U-Net affronta questo problema mediante **connessioni di salto** (*skip connections*) tra gli stadi corrispondenti dell'encoder e del decoder.

Aniché costringere tutte le informazioni a passare attraverso il collo di bottiglia, le mappe delle caratteristiche prodotte dall'encoder vengono trasferite direttamente al decoder in corrispondenza della medesima risoluzione. In genere, queste caratteristiche vengono concatenate con quelle del decoder prima di applicare le successive operazioni convoluzionali.

Il flusso informativo complessivo può quindi essere riassunto come segue:

$$\text{Ingresso} \rightarrow \text{Encoder} \rightarrow \text{Collo di bottiglia} \rightarrow \text{Decoder} \rightarrow \text{Maschera di segmentazione}$$

con connessioni di salto aggiuntive:

Caratteristiche dell'encoder $\rightarrow$ Caratteristiche del decoder.

Questo meccanismo consente al decoder di combinare due diversi tipi di informazioni:

- **caratteristiche di alto livello**, prodotte dagli strati più profondi, che aiutano a determinare se una struttura rappresenta un filamento;
- **caratteristiche spaziali fini**, ottenute dai primi strati dell'encoder, che aiutano a ricostruirne la posizione, la forma e i contorni.

Questa caratteristica è particolarmente rilevante per il nostro insieme di dati. I filamenti solari possono essere strutture sottili, allungate e irregolari; un'eccessiva perdita di informazioni spaziali durante il sottocampionamento potrebbe quindi causarne una ricostruzione errata o frammentata.

3.5 U-Net per la segmentazione binaria

Per l'attività considerata, U-Net è configurata in modo da produrre un singolo canale di uscita. Per ogni pixel (i, j) , la rete produce un logit z_{ij} , che può essere convertito in una probabilità mediante la funzione sigmoide:

$$p_{ij} = \sigma(z_{ij}) = \frac{1}{1 + e^{-z_{ij}}}. \quad (3)$$

La mappa di probabilità risultante presenta le stesse dimensioni spaziali della maschera di segmentazione in ingresso. È quindi possibile applicare una soglia t per ottenere la predizione binaria finale:

$$\hat{y}_{ij} = \begin{cases} 1, & p_{ij} \geq t, \\ 0, & p_{ij} < t. \end{cases} \quad (4)$$

Pertanto, il processo completo può essere riassunto come segue:

$$\text{Immagine solare} \rightarrow \text{U-Net} \rightarrow \text{Mappa di probabilità} \rightarrow \text{Maschera binaria dei filamenti}. \quad (5)$$

4 Esperimenti

4.1 Esperimento 1 – U-Net addestrata da zero con AdamW

Il primo esperimento definisce una baseline per i confronti successivi. La U-Net descritta nella Sezione 3 è stata inizializzata con pesi casuali, senza modelli preaddestrati.

Le immagini H-alpha sono state convertite in scala di grigi e ridimensionate da 2048×2048 a 512×512 pixel. Durante il training sono state applicate, sia all'immagine sia alla maschera, riflessioni orizzontali e verticali e rotazioni multiple di 90° . Lo split, effettuato sui nomi dei file univoci per evitare leakage tra annotazioni della stessa osservazione, è composto da 494 immagini per il training, 106 per la validation e 107 per il test interno.

La rete ha un canale di ingresso, 16 filtri nel primo livello e 1 942 289 parametri addestrabili. È stata addestrata per 20 epoche con batch size 2, AdamW ($\text{lr} = 10^{-3}$ e weight decay 10^{-4}) e una loss pari alla media tra Binary Cross-Entropy e soft Dice loss.

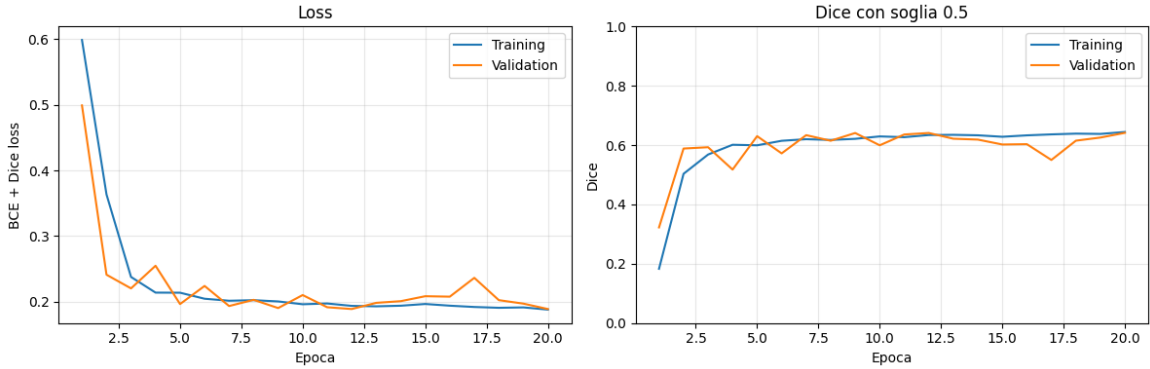


Figura 2: Andamento di loss e Dice sui set di training e validation nell'Esperimento 1.

Come mostrato nella Figura 2, la maggior parte del miglioramento avviene nelle prime epoche; in seguito le prestazioni di validation si stabilizzano con oscillazioni moderate. Il miglior Dice di validation con soglia 0,5 è 0,6415, ottenuto all'epoca 20.

La soglia per convertire la mappa di probabilità in maschera binaria è stata scelta esclusivamente sulla validation. Tra i valori compresi tra 0,10 e 0,90, la soglia 0,15 ottiene il Dice più alto (0,6462), indicando che una soglia più bassa recupera meglio i filamenti sottili.

Tabella 1: Risultati dell'Esperimento 1.

Metrica	Valore
Miglior Dice validation con $t = 0,5$	0,6415
Soglia selezionata sulla validation	0,15
Dice validation con soglia selezionata	0,6462
Loss sul test interno	0,2014
Dice sul test interno	0,6157

Il Dice sul test interno conferma che la baseline individua una parte significativa dei filamenti su osservazioni non utilizzate durante lo sviluppo. I risultati costituiscono il riferimento per l'Esperimento 2, nel quale verrà utilizzato SGD con momentum.

4.2 Esperimento 2 – U-Net con SGD, momentum ed early stopping

Il secondo esperimento confronta SGD con momentum con l’AdamW della baseline. Varia soltanto l’ottimizzatore: learning rate 10^{-2} , momentum classico 0,9 e weight decay 10^{-4} .

È stato inoltre introdotto un early stopping basato sul Dice di validation, la metrica più rappresentativa per una segmentazione binaria sbilanciata. Il training viene interrotto dopo otto epoche senza un miglioramento di almeno 0,001 nel Dice di validation. L’addestramento si è fermato all’epoca 18; il miglior modello è quello dell’epoca 10.

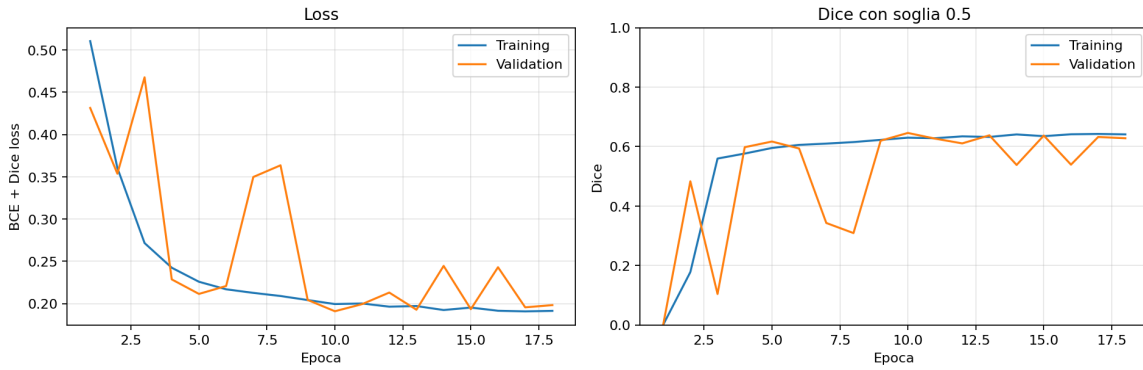


Figura 3: Loss e Dice dell’Esperimento 2 con SGD, momentum ed early stopping.

I risultati di validation sono confrontati nella Tabella 2. SGD ottiene un Dice leggermente superiore alla soglia standard 0,5, ma, dopo la selezione della soglia, le due configurazioni raggiungono lo stesso Dice di validation. La soglia ottimale per SGD è 0,70, mentre per AdamW era 0,15: gli optimizer producono quindi probabilità calibrate in modo diverso, senza un vantaggio netto in termini di sovrapposizione finale.

Tabella 2: Confronto tra AdamW e SGD sul validation set.

Metrica	AdamW	SGD
Miglior Dice con $t = 0,5$	0,6415	0,6457
Epoca migliore	20	10
Epoche completate	20	18
Soglia ottimale	0,15	0,70
Dice con soglia ottimale	0,6462	0,6462

Il test interno non è stato eseguito per l’Esperimento 2. La scelta tra le configurazioni viene effettuata sulla validation, lasciando il test per la valutazione finale del modello selezionato.

4.3 Esperimento 3 – U-Net con encoder ResNet18 pre-addestrato

Negli esperimenti precedenti il collo di bottiglia è la quantità di dati: con 806 record di training, l’encoder della U-Net deve imparare da zero sia le feature visive di basso livello sia quelle specifiche dei filamenti. L’ipotesi di questo esperimento è che le feature apprese su ImageNet – bordi, texture e strutture locali – costituiscano un punto di partenza migliore anche per le immagini solari, pur appartenendo a un dominio visivo molto diverso.

L’encoder della baseline è stato quindi sostituito con una **ResNet18 pre-addestrata su ImageNet** (*transfer learning* con *fine-tuning* completo): i quattro stadi residuali della rete

producono feature map a 64, 128, 256 e 512 canali, collegate tramite skip connections al decoder, che resta quello della baseline. Le immagini H-alpha sono in scala di grigi, mentre ResNet18 attende un ingresso RGB: il canale è stato replicato tre volte, senza modificare i pesi del primo strato convoluzionale. Il modello risultante ha 14 343 425 parametri addestrabili e occupa meno di 0,5 GB di memoria GPU durante il training, ampiamente sotto la soglia dei 5 GB indicata come bonus.

Rispetto alla baseline cambiano due sole scelte, rese necessarie dal fine-tuning: il learning rate è ridotto a 10^{-4} (AdamW), per non distruggere nelle prime epoche le feature pre-addestrate, e i canali di ingresso diventano tre. Split, augmentation, loss, batch size, risoluzione e protocollo di early stopping sono invariati.

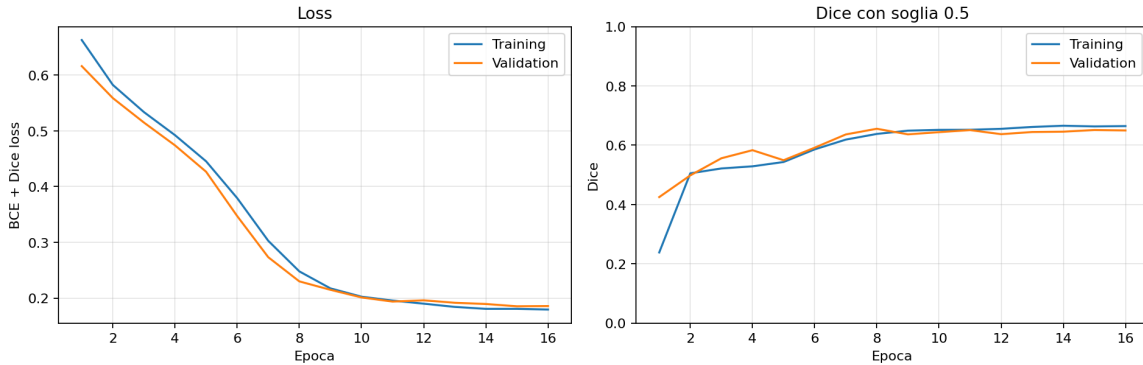


Figura 4: Andamento di loss e Dice nell’Esperimento 3 (encoder ResNet18 pre-addestrato).

Come mostrato nella Figura 4, il fine-tuning converge molto più rapidamente delle configurazioni addestrate da zero: il miglior Dice di validation con soglia 0,5 è 0,6552, raggiunto già all’epoca 8, e l’early stopping interrompe il training all’epoca 16. Il guadagno rispetto alla migliore configurazione da zero è di circa un punto Dice (0,6552 contro 0,6457), con meno della metà delle epoche effettive e curve di training e validation ravvicinate, senza segni di overfitting.

L’effetto più rilevante riguarda però la **calibrazione delle probabilità**. La curva soglia–Dice è quasi piatta: il Dice di validation resta compreso tra 0,649 e 0,655 per tutte le soglie tra 0,10 e 0,90, con massimo 0,6554 a $t = 0,45$. Negli esperimenti 1 e 2 la soglia ottimale si trovava agli estremi dell’intervallo (0,15 e 0,70) e la sua scelta era quindi critica; con l’encoder pre-addestrato la soglia standard 0,5 è già quasi ottimale. Questo comportamento è particolarmente desiderabile in prospettiva della submission Kaggle, dove la soglia deve essere fissata a priori senza accesso alla ground truth. I risultati numerici di tutti gli esperimenti sono raccolti e confrontati nella Tabella 3.

Dal controllo qualitativo sulla validation (predizioni salvate in `experiments/03/plots`) emergono ancora due comportamenti tipici: falsi positivi di piccole dimensioni in prossimità del bordo del disco e mancato riconoscimento dei filamenti più deboli e sottili. Entrambi i problemi sono coerenti con la perdita di dettaglio introdotta dal ridimensionamento a 512×512 e motivano le modifiche proposte di seguito.

4.4 Esperimento 4 – Training su patch ad alta risoluzione

Il controllo qualitativo dell’Esperimento 3 ha evidenziato errori concentrati sui filamenti più sottili e deboli. L’ipotesi è che il collo di bottiglia non sia più la capacità del modello, ma la **perdita di dettaglio introdotta dal ridimensionamento**: un fattore di scala $4\times$ (da

2048×2048 a 512×512) riduce un filamento di 6 pixel a meno di 2 pixel e cancella quelli più sottili sia nell'immagine sia, per effetto del resize nearest-neighbor, nella ground truth stessa.

In questo esperimento il training avviene su **crop casuali** 512×512 **estratti dalla risoluzione nativa**, senza alcun ridimensionamento: le strutture sottili conservano la loro larghezza originale sia in ingresso sia nella maschera. Poiché i filamenti occupano una frazione molto piccola dell'immagine, un crop completamente casuale conterrebbe spesso solo sfondo; il campionamento è quindi bilanciato: nell'80% dei casi il crop contiene almeno un pixel di filamento (massimo 10 tentativi), nel restante 20% è casuale puro e include sfondo, bordo del disco e rumore, preservando esempi negativi. Modello, ottimizzatore, loss, augmentation e split sono identici all'Esperimento 3: l'unica variabile modificata è la risoluzione effettiva dell'ingresso.

Anche validation e test devono operare alla risoluzione nativa: valutare il modello su immagini ridimensionate lo esporrebbe a un ingresso fuori distribuzione. Ogni immagine è divisa in una griglia 4×4 di tasselli da 512×512 , le probabilità predette sono ricomposte in una mappa 2048×2048 e poi ridotte a 512×512 : il Dice calcolato su queste mappe ridotte usa la stessa ground truth degli esperimenti precedenti, mantenendo la **confrontabilità diretta**; il Dice alla risoluzione nativa è riportato come informazione aggiuntiva. La selezione della soglia e l'early stopping usano il Dice a 512×512 . Il costo di memoria resta invariato, perché la dimensione dell'ingresso della rete non cambia; aumenta solo il tempo di valutazione (16 forward per immagine).

Si noti infine una differenza metodologica: in ogni epoca ciascuna immagine di training contribuisce con un solo crop casuale, quindi la copertura del dataset per epoca è inferiore, ma la diversità dei campioni è maggiore e agisce come ulteriore forma di augmentation.

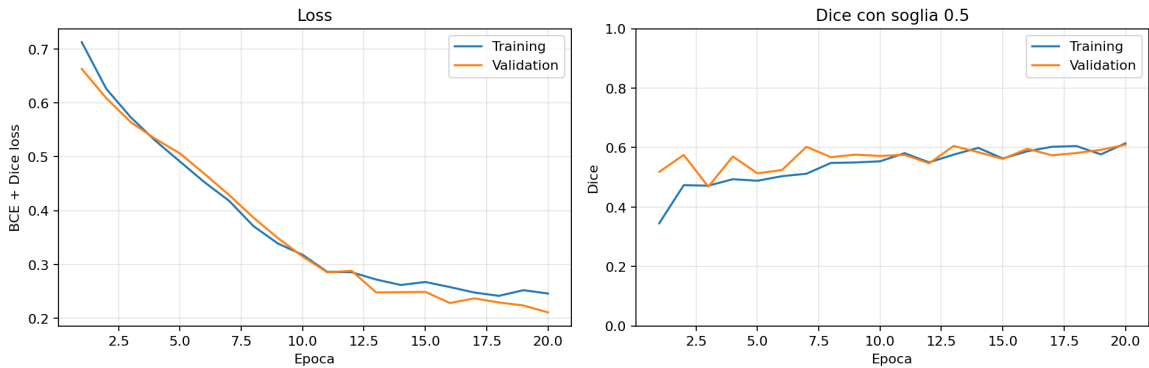


Figura 5: Andamento di loss e Dice nell'Esperimento 4 (training su crop alla risoluzione nativa).

L'ipotesi **non è confermata**: con un Dice di validation di 0,6093 a soglia 0,5, il modello addestrato su crop nativi risulta inferiore di circa quattro punti e mezzo a quello addestrato su immagini ridimensionate (0,6552). Il divario resta invariato dopo la selezione della soglia (0,6127 con $t = 0,20$) e il Dice calcolato alla risoluzione nativa (0,6130) non mostra alcun beneficio dalla maggiore risoluzione effettiva. Il confronto qualitativo sulla validation conferma: gli errori sui filamenti sottili osservati nell'Esperimento 3 non sono stati recuperati.

L'analisi delle curve nella Figura 5 indica però che il fattore limitante non è la capacità del modello, ma la **copertura del dataset**: con un solo crop casuale per immagine, a ogni epoca il modello osserva circa un sedicesimo dei pixel di ciascuna osservazione e, a parità di numero di epoche, il volume di dati effettivamente visto è una frazione di quello degli esperimenti precedenti. Tre evidenze supportano questa lettura: il Dice migliore si ottiene all'**ultima** epoca, l'early stopping non interviene mai, e sia la loss di validation sia il Dice di training stanno

ancora migliorando al termine del training. Il modello è quindi sotto-allenato, non incapace di sfruttare la risoluzione nativa.

Un secondo effetto riguarda la calibrazione: il campionamento bilanciato dei crop (80% contenenti filamento) sovrarappresenta i pixel positivi rispetto all’immagine intera e la soglia ottimale scende a 0,20; la curva soglia–Dice resta comunque ampia e piatta (0,60–0,61 su tutto l’intervallo), quindi la scelta della soglia non è critica.

Tabella 3: Confronto tra le configurazioni sul validation set.

Metrica	Exp. 1	Exp. 2	Exp. 3	Exp. 4	Exp. 5
Miglior Dice con $t = 0,5$	0,6415	0,6457	0,6552	0,6093	0,5921
Epoca migliore	20	10	8	20	10
Epoche completate	20	18	16	20	10
Soglia ottimale	0,15	0,70	0,45	0,20	0,35
Dice con soglia ottimale	0,6462	0,6462	0,6554	0,6127	0,5938
Dice nativo con soglia ottimale	–	–	–	0,6130	0,5965

Per completezza, su questo esperimento è stata eseguita anche la valutazione sul test interno: Dice 0,6022 con $t = 0,20$ (0,6039 alla risoluzione nativa), coerente con la validation. La selezione del modello finale resta basata esclusivamente sulla validation; le misure di test sono riportate a solo scopo di analisi.

4.5 Esperimento 5 – Fine-tuning a due stadi su crop nativi

L’Esperimento 4 ha mostrato che il training su crop nativi soffre la scarsa copertura del dataset: a parità di epoche il modello osserva una frazione dei pixel e non converge. La contromisura testata qui separa i due obiettivi di apprendimento: il **contesto globale** si apprende sulle immagini ridimensionate (intera osservazione per campione), il **dettaglio spaziale** sui crop nativi. Si parte quindi dai pesi dell’Esperimento 3 – il migliore sulla validation – e il training prosegue per un massimo di dieci epoche sui crop 512×512 nativi, con learning rate ridotto a 10^{-5} , un ordine di grandezza sotto quello del training originale, per adattare i pesi al nuovo dominio di ingresso senza distruggere quanto già appreso; l’early stopping opera con pazienza 4. Campionamento dei crop, augmentation, loss, split e protocollo di valutazione a tasselli sono identici all’Esperimento 4: l’unica variabile modificata è l’inizializzazione.

In questa configurazione è stata anche ottimizzata l’efficienza del training: la decodifica delle immagini ad alta risoluzione e la rasterizzazione dei poligoni dominano il tempo di epoca, quindi immagini e maschere sono conservate in memoria dopo il primo caricamento; dalla seconda epoca il throughput è limitato soltanto dalla GPU.

Il fine-tuning a due stadi adatta progressivamente il modello al dominio dei crop nativi – il Dice di validation sale da 0,458 a 0,5921 in dieci epoche – ma non raggiunge né l’Esperimento 4 (0,6093) né, soprattutto, l’Esperimento 3 (0,6552), come riassunto nella Tabella 3. Due osservazioni emergono dall’analisi delle curve nella Figura 6.

In primo luogo, anche in questo esperimento il miglior Dice si ottiene all’**ultima** epoca e l’early stopping non interviene: la curva di validation è ancora in crescita al termine del training. **Tutti** i training su crop (Esperimenti 4 e 5) sono terminati mentre il modello stava ancora migliorando, confermando che la copertura ridotta del dataset richiede budget di training molto superiori a quelli sufficienti per le immagini ridimensionate. In secondo luogo, il learning rate

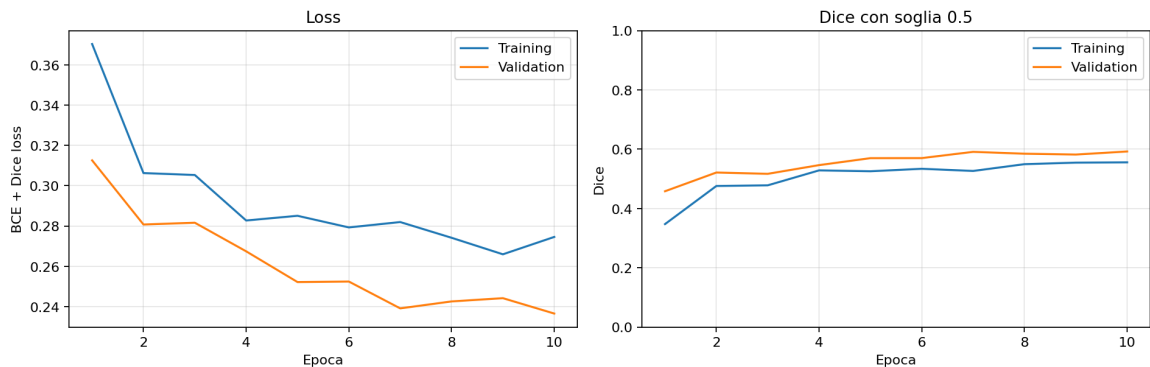


Figura 6: Andamento di loss e Dice nell’Esperimento 5 (fine-tuning a due stadi su crop nativi).

10^{-5} si è probabilmente rivelato eccessivamente prudente per il riadattamento al nuovo dominio di ingresso: dieci epoche a questo passo non bastano a colmare il divario.

Va infine osservato che i pesi dell’Esperimento 3, sottoposti a inferenza a tasselli sulla risoluzione nativa, partono da un Dice inferiore a 0,46: il modello addestrato su immagini ridimensionate non è in grado di operare direttamente alla risoluzione originale. Se la submission Kaggle è valutata a piena risoluzione, il confronto rilevante diventa il Dice nativo, sul quale gli Esperimenti 4 (0,6130) e 5 (0,5965) sono i candidati migliori nonostante il divario sulla metrica a 512×512 . La verifica della risoluzione attesa dalla competizione è dunque un passaggio obbligato prima della selezione del modello finale.

4.6 Analisi degli errori

Il controllo qualitativo sulle predizioni mostra due difetti ricorrenti: i filamenti predetti appaiono in parte più spessi della ground truth e sono presenti piccoli falsi positivi, soprattutto vicino al bordo del disco. Per quantificare questi difetti abbiamo analizzato le predizioni del modello dell’Esperimento 3 sulla validation alla soglia selezionata ($t = 0,45$). Precision e recall medie risultano quasi uguali (0,669 e 0,680) e il rapporto tra area predetta e area annotata ha mediana 0,997: non esiste quindi un bias sistematico di ispessimento, e l’effetto osservato visivamente dipende dall’alone di probabilità attorno ai contorni. I falsi positivi sono invece componenti connesse molto piccole: mediana di 7 pixel, con il 90% dei casi sotto i 64 pixel.

Abbiamo quindi verificato se un semplice post-processing potesse migliorare il Dice di validation. La rimozione delle componenti connesse sotto i 16 pixel non cambia il risultato (0,6559 contro 0,6554), mentre soglie più aggressive o l’erosione morfologica peggiorano nettamente la metrica (fino a 0,51): anche la ground truth contiene infatti molti filamenti di pochi pixel a 512×512 , e filtri di questo tipo eliminano più veri positivi che falsi positivi. Il post-processing morfologico è stato quindi scartato.

Per capire dove si concentrano gli errori residui abbiamo infine misurato l’accordo tra gli annotatori del dataset. 296 file possiedono due o tre annotazioni indipendenti: sulle 598 coppie, il Dice medio tra due annotazioni della stessa immagine è 0,62 (mediana 0,65, 10°–90° percentile 0,43–0,77). I filamenti coprono in media lo 0,38% dei pixel e le strutture più deboli sono annotate in modo diverso da annotatori diversi; gli errori residui del modello si concentrano proprio su queste strutture ambigue e sui contorni. Questo suggerisce che il margine di miglioramento ulteriore sul dataset è limitato, e che la selezione del modello finale non deve inseguire variazioni di Dice dell’ordine di pochi millesimi.

4.7 Modifiche proposte

Le misure della Sezione 4.6 guidano le scelte finali. Poiché le immagini di training e di test hanno risoluzione 2048×2048 , assumiamo che le maschere di submission debbano essere prodotte alla stessa risoluzione: il confronto rilevante è quindi il Dice nativo e il candidato finale è la pipeline a crop. Resta da eseguire un ultimo training con budget adeguato, proseguendo dal checkpoint dell'Esperimento 4 per 30–40 epoche con learning rate 10^{-4} : la cache introdotta nell'Esperimento 5 rende il costo sostenibile e tutte le curve degli esperimenti su crop terminano ancora in crescita. Per la submission, prodotta con inferenza a tasselli, si applicherà la **test-time augmentation** (media delle probabilità su versioni ruotate e riflesse della stessa immagine) e, se i tempi lo consentono, un **ensemble** di modelli addestrati con seed diversi. Il post-processing morfologico è stato testato e scartato, come riportato nella Sezione 4.6.